Function Identification in Android Binaries with Deep Learning

Aizaz Sharif
Dept. of Computer Science
FAST NUCES
Peshawar, Pakistan
p177003@nu.edu.pk

Mohammad Nauman
Dept. of Computer Science
FAST NUCES
Peshawar, Pakistan
mohammad.nauman@nu.edu.pk

***Abstract*—Application security support has become a preference for the enterprise as cybersecurity threats have transferred from the network perimeter to the application layer in modern years. To ensure that the software is secure, organizations must test it before purchase or deployment and identify any flaws that may expose the organizations to vulnerabilities. Binary code analysis is a new method for application security testing and is transforming software security. Binary analysis aids in many important applications including automatically fixing vulnerable software and malware detection. Plenty of research has been done to improve binary analysis using datasets related to different computer platforms, new compilers, and new optimization techniques. However, there is a vast majority of Android users and since it is an open-source platform it is equally vulnerable to similar attacks as well. In this research, we propose to implement deep neural networks to solve an essential yet difficult problem in binary analysis. We solve the problem of function boundary identification, a pivotal first step in a lot of binary analysis techniques. Neural networks have experienced a renewal in the past few years, achieving breakthrough outcomes in various fields such as object detection, language translation, and speech recognition. Yet no specific research has explored their utility in Android binary analysis. We exhibit that convolutional neural networks can identify functions in binaries with greater accuracy than the current state-of-the-art methods. Our model will be trained on a dataset of bytecode extracted from Android binaries. The proposed methodology is tested and evaluated on the Drebin Malware Dataset which contains Android Malware applications coming from various malware families. With the presented model, we achieved an overall precision of 0.75, recall of 0.79 and the f1 score of 0.76 in the testing phase.**

***Index Terms*—Function Identification, Function Boundary, Malware Detection, Binary Analysis, Deep Learning, Convolutional Neural Networks**

I. Introduction

Binary code Analysis [1] is the technique used in application security which assists in many purposes e.g. control-flow integrity, malware detection, vulnerability detection, prevention, and many more. It is because of the binary code analysis which gives the start to other useful computer and mobile security applications. It is mainly involved where the source code is unavailable to be compared or evaluated with the existing tested stripped binaries. Whether it is malware to detect or protocol reverse-engineering, all of these tasks become difficult to operate straight on the binaries.

The main challenge faced by the binary analysis is the generation of binaries using more than one operating system, optimizations, and compilers. This leads to the lack of high-level semantic structure among the stripped binaries since the information regarding this is removed by the compilers.

Functions (also known as methods or procedures-depending on the language) are the basic but also the key structural piece that exists among every program out there. These functions become hard to detect when we try to see the sequence of machine assembly instructions in the given binaries. It is, therefore, the main focus of all the application security areas that depend on binary analysis to first correctly identify the functions and also recovers them properly. This is known as the problem of *function identification*. There are many examples in security where function identification can help. For example, it can help out constructing control flow integrity graphs, and also it aids debuggers and decompilers since they require the information of where the function is located [2].

A lot of work has been done in the past years for the task of identifying function boundaries inside stripped binaries from a different source of datasets. Although, the research so far has not been initiated in the field of Android security for the binary analysis. In this paper, we will be introducing the problem of function identification in Android binaries by using deep neural networks to correctly identify each instruction.

The research in this paper will introduce the function identification problem using the advantages of neural networks. Neural networks were first introduced to the world in the 1940s and since then it has gone through some marvelous advancements in its research. Over the time it went through its ups and downs but in the 21st century due to the availability of computational power and big data, we saw the new era of Deep Neural Networks which comes under the umbrella of Machine Learning and Artificial Intelligence. Massive amounts of data and processing power provided researchers with the perfect opportunity to drive breakthroughs in the areas of Computer Vision, Speech Recognition, and many more [3].

Neural networks come out with some amazing features like the ability to learn the representation of original data with fewer steps of feature engineering, which is the core issue of every dataset out there. Looking at the success rate of the neural networks among the vast variety of problems,

the question arises whether we can introduce the problem of function identification in the Android Binaries. Nevertheless, the literature proves that using supervised learning by Rosenblum [4] and Bao [2] they were able to achieve better results, and Richard Shin [5] in 2017 further expanded the work by solving this problem using Recurrent neural networks [6] which is one of the core algorithms from deep neural networks.

With our submitted solution, we are able to build a Convolutional Neural Network (CNN) [7] model that trains the network over bytecode instructions taken from Android binaries to identify whether a given instruction over a sequence of time is Function Boundary start or not. CNN is a much appropriate choice in this research since the dataset we deal with contains a lot of instructions, and also it increases the training and validation speed of whole architecture as compared to other deep models. We evaluate our results by the end of our experiment by using our own dataset that will be created using a subset of Malware Android application files. As a result, the convolutional neural network performed much faster than the recurrent neural networks solution (specifically Bi-LSTM) proposed by [5]. The number of instructions in our dataset is over 25 million but still, our experiments worked well and the model completed its training and testing over instructions within a few hours.

Binary analysis as discussed before contains many fields under its umbrella, and so we take Android Malware detection as our main motivation since it also requires accurate identification of function boundaries.

In the rest of the paper, we first define the scope of the problem. We then explain the necessary background required to understand the neural network pipeline and also describe the specific deep neural architecture that we chose to build for our scope of the experiment. We provide the results during the model evaluation along with the related literature review for the function identification in binaries. The contributions of our research in this paper are summarized as:

- We introduce the problem of function boundary detection in the area of Android security.
- We also create a dataset containing opcodes of instructions with their embedding vectors along with a class defining function start.
- We built the CNN Deep Learning Architecture pipeline to train and validate the dataset and achieve an overall F1 score of 0.76.

II. Problem Definition

Here we first define the scope of the research in order to accurately address the function identification problem in this paper. Moreover, we provide a formal yet known definition of function identification.

A. Scope

We relate our work by doing our experiments over bytecode instructions derived from given Android applications. We will ignore various information that is typically related to the source code itself, for instance, constant strings, global and local variables as few of them. We use the application (.apk) as an executable having code which contains a sequence of bytes. Each instruction contains one opcode byte that belongs to Dalvik Instruction Set [8]. The main thing to consider is that our dataset contains opcodes instructions. This way working in terms of opcodes allows us to easily work with Android bytecodes compiled by Java compiler.

B. Task definition

Let us take the assumption of having access to the bytecode of an Android application, but with the code, there is no information of functions $f_1,...,f_n$. We define the following tasks:

- Given bytecodes, find the opcodes instructions to locate the function start boundary, i.e., $\{f_{1,1},...,f_{n,1}\}$.
- Given bytecodes, find the opcodes instructions to locate the function end boundary, i.e., $\{f_{1,l_1},...,f_{n,l_n}\}$.
- Given bytecodes, find the opcodes instructions to locate the function start and ending boundary, i.e., $\{(f_{1,1},f_{1,l_1}),...,(f_{n,1},f_{n,l_n})\}$.
- Given bytecodes, derive the number of functions in the apk file, together with the total number of instructions that makes up each function as a whole. This can be represented as $\{(f_{1,1},f_{1,2},...,f_{1,l_1}),...,(f_{n,1},f_{n,2},...,f_{n,l_n})\}$.

As we can depict, the problem of function start and end identification is considered a subset of function boundary identification, while general function identification with the total count of functions is the superset of all the mentioned tasks in this area. For the sake of the experiment, our goal is to target the first one only, that is function start boundary detection, and therefore the remaining will be left for future contributions.

III. Background

In this section, we provide a brief explanation of the neural network and of parts of the neural networks together with the process of training as a whole. Our particular focus is on the different variations of deep neural networks, more specifically on convolutional neural nets which is the desired model we use in our architecture pipeline to classify the instructions.

A. Neural networks

A neural network is a model used for creating an approximation over a set of information for the sake of learning for future data. The main model of the neural network is inspired by the human brain. Such a model takes input as either a real-world data or pre-defined features of the real-world data, and it learns to represent for predicting the data that is unseen so far.

B. Input layer

The very first layer of any neural network architecture is called the input layer. This layer is provided with the data in the form of text, images, audio signals, and etc. Usually, in vanilla machine learning models, the features engineering is performed to determine the features that will be used by the input layer to pass it on to the proceeding hidden layers.

C. Hidden layer

This is where we apply the most prominent advancements of the neural networks. Hidden layers perform matrix multiplication and passed through activations on repeat over the number of neurons in each hidden layer until it reaches the output layer for prediction. The number of hidden layers can be increased according to the problem complexity and this led to the main form of deep neural networks which are right now the backbone of artificial intelligence. A hidden layer can be represented as:

$$h(x) = \Theta_0 + \Theta_1 x$$

Where *h* defines the current hidden layer nodes and *x* refer to the input from the previous layer. θ_0 and θ_i are the weight parameters used for the learning of model.

D. Activations

Activation functions are used inside each neuron of the hidden and output layer and it is also termed as squashing function. Activation plays an important role in the feed forward networks to decide which neurons should be activated based on the input values coming from the previous layer. And just like the name suggests, it squashes the value to a certain range depending on the threshold inside the activation function.

E. Output layer

The output layer is the last layer of every deep neural net out there which serves the purpose of classifying the data fed into the input layer (in supervised learning). The decision is made on the output neurons using sigmoid or softmax functions and this is the layer where backpropagation is applied to fit the model.

F. Regularization and hyperparameters

Regularization is applied where we provide nonlinear and high complex data but the model instead starts overfitting and therefore it avoids that by restricting the model to learn features that are too big. Hyperparameters [9] on the other hand is the model settings that we tune over time to achieve the best results out of architecture.

G. Sequence Models

Since we will be dealing with opcodes, they can also be treated as some form of text into the state-of-the-art deep learning sequence models. Works including [5] have utilized such algorithms to perform function boundary detection in recent years as we have seen in the literature. There are many forms of modeling as a Sequence of text or instructions, and commonly sequences are modeled using Recurrent Neural Networks (RNN) [6]. The model contains two input sources to deal with, the present timestamp data and the previous recent most timestamp information. Both are being combined to learn the patterns in the unseen data just like we perform ourselves in real-life scenarios. There are issues connected to RNN in recent times such as vanishing gradients [10], therefore certain variants have been proposed such as Long Short Term Memory (LSTM) [11] and are commonly used as deep learning models as an alternative.

LSTM contain information outside the normal process of the famous RNNs. LSTM have blocks which are effective of passing information through them and they are known as gates. A gate is a collection of mathematical computations similar to what we see in neural networks but with a sigmoid activations along with multiplication performed in a point wise manner. These can be outlined as follows:

$$h^t = \sigma(Wx^t + Uh_{t-1}) \tag{A}$$

$$remember^t = \sigma(W_r x^t + U_r wm_{t-1}) \tag{B}$$

$$ltm^t = \sigma(W_l x^t + U_l wm_{t-1}) \tag{C}$$

$$save^t = \sigma(W_s x^t + U_s wm_{t-1}) \tag{D}$$

$$focus^t = \sigma(W_f x^t + U_f wm_{t-1}) \tag{E}$$

Here *h* presents the hidden layer of the current state, *h_t-1* shows the activations of the previous hidden layer state and σ is the activation function. *W* and *U* are the weights learned through backpropagation of the network. Each hidden layer *h* is calculated by current weighted matrix on hidden layer for a given input *x* and addition to previous hidden layers values. Hence the base model of LSTM can learn both character-by-character and Sequence-by-Sequence modeling.

H. Convolutional Neural Networks

LSTM has been used recently to identify the function boundaries within computer stripped binaries, but in our experiments, they failed to perform that well due to the increased size and nature of the dataset. Therefore, we came up with a different approach where after a lot of experiments we built a complete Convolutional Neural Network (CNN) [12] using 1D convolutions over the input features. CNN's are mostly used for the tasks of image processing, autoencoders, and Generative Adversarial Networks. Recently they have been introduced to sentence classification and surprisingly it produced significant results. Our final model will be based on CNN operations which will be discussed in detail in Section IV.

IV. Proposed Architecture

In this section, we look at the model architecture that we built using the background of the neural networks mentioned in Section III to perform the objective of function identification using Android binaries.

A. Proposed Methodology

Function Boundary detection is one of the pioneer steps in binary analysis and so thus in related fields like malware analysis. Accurate identification of function start instructions is much needed in binary code analysis and security modules to complete their desired tasks. In this research, we create a deep neural architecture pipeline that will go through each instruction to determine whether they are the function boundary start or not. The proposed method contains 4 essentials steps

(as shown in the Figure 1, namely, 1) Reverse engineering Android applications into bytecode instructions, 2) Preprocessing of Bytecodes to create a dataset, 3) Deep Convolutional Neural Network, and 4) Function Boundary detection as a classification layer. The following paragraphs include one step at a time explanation of all the significant processes that are involved in the proposed methodology.

B. Data Preprocessing

The input required for this particular step is the bytecode raw instructions extracted from Android Malware Dataset. Each instruction contains hex of one opcode followed by one or more operands. These bytecodes need to be passed through the stage of preprocessing such that it is in a ready state before it is fed to the model. Here are the following steps through which the bytecode instructions of an Android app face preprocessing:

1) Removal of Operands
2) Labelling of opcode instructions
3) Embeddings

1) Removal of Operands: Our dataset contains 100 Android Malware Instructions, and inside each application while reverse-engineered we receive an uneven number of thousands of bytecode instructions. The very first step required to do is to remove all the operands from each instruction to limit the data to opcodes only which are total 256 as mentioned in the documentation of Android Developers [8].

2) Labeling of opcode instructions: After the first step, the next thing required in the process of dataset creation is providing a class to each instruction opcode. In our scenario, the function start instruction is labeled as 1 while the rest are defined as 0. We have limited the scope of research to function boundary start only. The pictorial representation of these two stages can be seen in the Figure 2.

3) Embeddings: Since the opcodes do not belong to any language vocabulary, finding word embeddings out of the famously known models such as Glove [13] and Word2Vec [14] is not prudent. As a solution, we will be utilizing Chars2Vec [15] as a language model (as shown in Figure 3) which is based on symbolic embeddings of words. The model is written in python and represents each sequence of symbols of arbitrary length with a fixed-length vector. In our case, the length of embedding will be fixed to 50. This model is substantially more helpful in a case where words (e.g. opcodes) do not belong to the vocabulary of a semantic language model.

C. Classifier Weights Training

The second stage that comes across after preprocessing is the model training for the calculation of weights. The sole purpose of this stage is to use these trained weight values for the identification of function boundary detection.

The model weights comprise of data preprocessing and the process of training a deep convolutional model as depicted in the Figure 1. Data preprocessing is required so that we can filter out opcodes, label each instruction, and obtain the final vector representations of each opcode that can be fed to the deep neural network as thoroughly explained in the section IV-B. The vectors along with the labels are given to the model and at each step, the ground truth is compared to predictions to improve and evaluate the model performance.

D. CNN Architecture

There is tons of ongoing research related to text classification and many have even released their model architectures based on deep learning techniques. Usually, the Recurrent Neural Networks [16] and LSTM [11] are considered to be the first priority when it comes to dealing with textual data whether in classification or clustering, but in our case, since there are around 25 million instructions, the performance of LSTM has not shown promising results. After several tries, we have adopted the Convolutional Neural Network [7] model. The article is famous for its unique approach to the classification of any textual information using CNN. We, therefore, adopted the model and tweaked the parameters according to our needs to perform function boundary detection. The proposed CNN is a fully connected deep network which only contains the extraction of feature maps from input vectors and pooling operations.

The CNN model as displayed in the Figure 4 can be divided into multiple stages. In the first stage, the matrix of input embeddings $\times$ output dimensions is passed through 3 separate convolution operations. Each convolution will have a specified number of filters and the kernel size as well. While in the 2nd stage, each of the 3 outputs of convolutions is passed through max-pooling operations individually. Finally, they are concatenated into a single layer which is flattened to connect to the rest of the hidden and output layer. The details mentioned above can be clearly compared in the Figure 4

1) Initialization: Choosing a suitable weight initializer is an important part of any model training process. The primary reason is, to obtain convergent weights towards the dataset we provide. A lot of various weight initializers are present in deep learning frameworks (Keras [17] in our case) and therefore for the safe side, we are choosing default Keras.initializers.Constant [17] as the initial model weight parameters.

2) Activation Function: There are numerous types of activation functions throughout the deep learning research community and the selection of it is mostly based on what is previously chosen in a similar scenario as well as how it performs in our situation. Therefore in our architecture, we chose Rectifier Linear Unit (ReLU) [18]. We have tested other variants of the ReLU but the results are not significant as compared to ReLU.

3) Convolution: Convolution inside the deep neural model is divided into 3 separate blocks, where each block applies similar repeating steps. Each block applies convolutional layers of different filters with different kernel sizes and a max-pooling operation. Pooling operation is used in CNN to reduce the computational load of the neural network on the GPU. The pooling process should not be used in starting layers

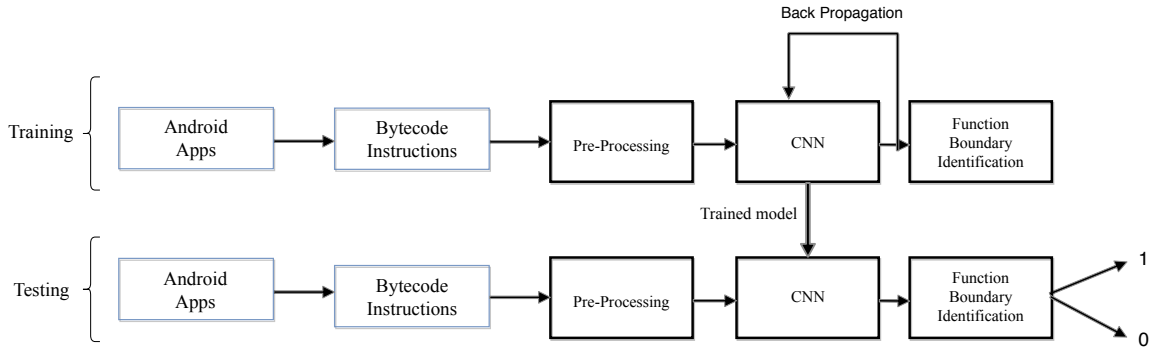

Fig. 1. Representation of step by step processes involved in the proposed method

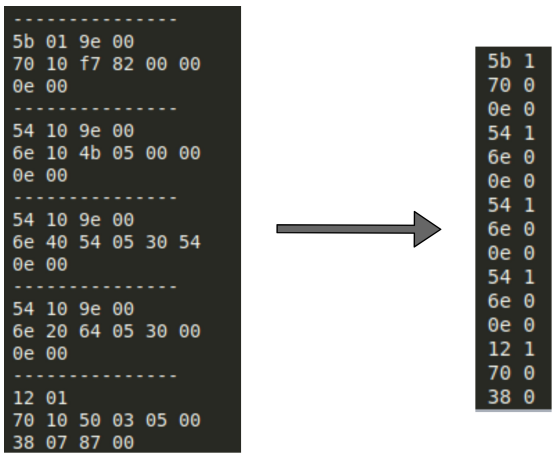

Fig. 2. Dataset Preprocessing

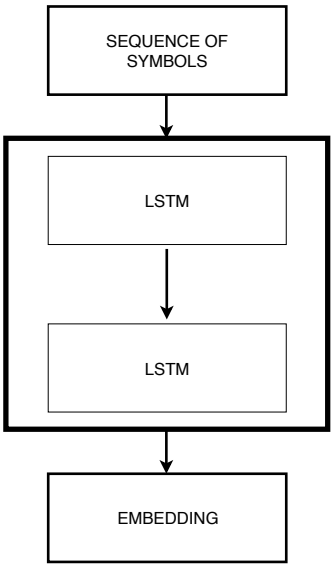

Fig. 3. Chars2Vec Architecture

as important information in the form of features might be lost. In our CNN architecture, we would be exploiting Max Pooling [19], along with the convolutional layer where the number of filters in each block is 16 and the size of the kernel is 3 and 5 respectively.

4) Dense Layers: After the convolution and max-pooling are done, each of their output tensors is concatenated into a single tensor and it is then reshaped to 1-dimensional flatten layer abstract representation of input layers. This layer is then connected to 2 fully connected hidden layers and the final output layer. The output layer contains 2 nodes, each representing the function boundary class as 1 and 0 after going through a sigmoid [20] activation function.

5) Regularization: Regularization in deep learning is applied to avoid the overfitting of any data during the training of the model weights. The main purpose of regularizations comes from the concept of how much a model is able to generalize itself. There are many techniques that are constantly applied to tackle this common issue, and in our case, we will be utilizing Batch Normalization [21] as a regularization parameter with the default values.

6) Loss Function: Backpropagation [22] is the essential step of any neural network and deep network learning, and the main goal of backpropagation is to minimize the cost function against the ground truth. A lot of research is done for establishing the various cost functions which work in many scenarios and therefore every framework out there is continuously updating themselves with the changes. We will be using binary crossentropy [23] since we are focused on a binary classification problem.

7) Minimization Function: The sole purpose of the training in deep learning is to lower the cost of the model weights throughout the network by going through many epochs. This is mainly achieved by reaching a global minimum. Therefore, in order to fully optimize the model, we have tried multiple techniques such as the famous Stochastic Gradient Descent (SGD) [24] but in the end, we have settled with the Adam optimizer [25].

E. Function Boundary Detection

For function boundary detection, each instruction is passed to the CNN after data preprocessing to generate final output as

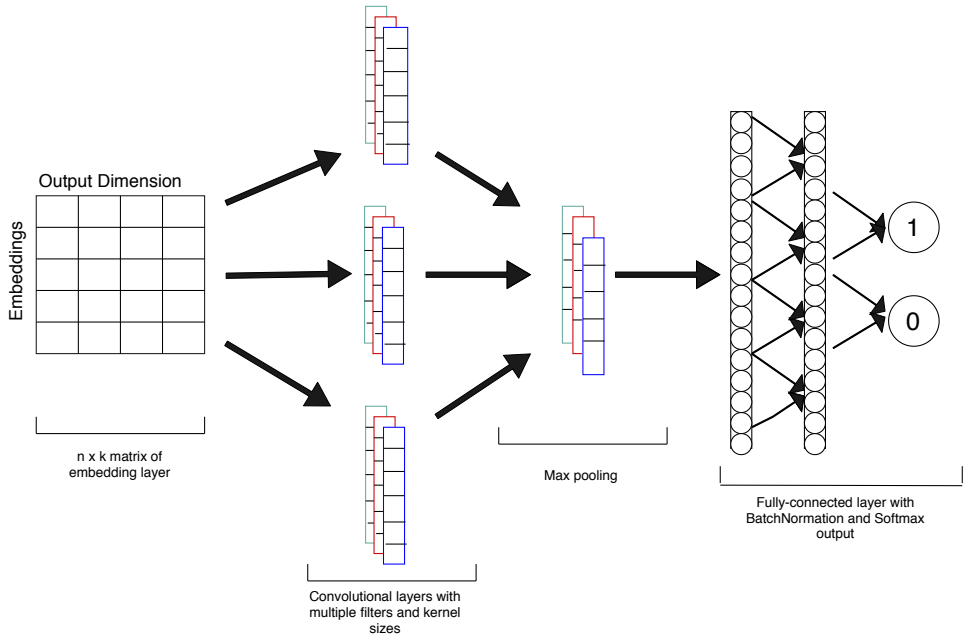

Fig. 4. Diagrammatic view of all the convolutions layers, filters, pooling, and connected layers of our proposed CNN architecture

well as the weights that are trained as told in the section IV-C. The output tensor we receive at the end of the network can be shown as:

$$output_tensor = (batch\ size,\ no.\ of\ classes) \quad (1)$$

In Equation 1, the batch size is defined at the start of the model training which in our case is set to 2048, and the number of classes is predicted against each instruction in the whole batch of opcodes.

1) Architecture Summary: Table I demonstrated is showing the overall picture of the architecture that we are using for the task of function boundary identification. This table provides a clear description of every layer and the operations that will be performed on each layer as well from start to finish. The model is discussed in Section IV-D as well.

TABLE I
ARCHITECTURE OF PROPOSED CNN MODEL FOR THE DETECTION OF FUNCTION BOUNDARY

Layer	Operation	Output Shape	Parameters	Connected to
Layer-1	Input	(None,50)	0	-
Layer-2	Embedding	(None,50,150)	26850	Layer-1
Layer-3	Reshape	(None,50,150,1)	0	Layer-2
Layer-4	BatchNorm	(None,50,150,1)	4	Layer-3
Layer-5	Conv2d(3*3)	(None,48,1,16)	7216	Layer-4
Layer-6	Conv2d(5*5)	(None,46,1,16)	9616	Layer-4
Layer-7	Conv2d(5*5)	(None,46,1,16)	12016	Layer-4
Layer-8	BatchNorm	(None,48,1,16)	64	Layer-5
Layer-9	BatchNorm	(None,46,1,16)	64	Layer-6
Layer-10	BatchNorm	(None,46,1,16)	64	Layer-7
Layer-11	ReLU	(None,48,1,16)	0	Layer-8
Layer-12	ReLU	(None,46,1,16)	0	Layer-9
Layer-13	ReLU	(None,46,1,16)	0	Layer-10
Layer-14	Max-Pooling	(None,1,1,16)	0	Layer-11
Layer-15	Max-Pooling	(None,1,1,16)	0	Layer-12
Layer-16	Max-Pooling	(None,1,1,16)	0	Layer-13
Layer-17	Concatenate	(None,3,1,16)	0	Layer14 - Layer-16
Layer-18	Flatten	(None,48)	0	Layer-17
Layer-19	Dense	(None,15)	490	Layer-18
Layer-20	BatchNorm	(None,15)	40	Layer-19
Layer-21	ReLU	(None,15)	0	Layer-20
Layer-22	Dense	(None,10)	480	Layer-21
Layer-23	BatchNorm	(None,10)	40	Layer-22
Layer-24	ReLU	(None,10)	0	Layer-23
Layer-25	Dense	(None,2)	22	Layer-24

V. EXPERIMENTAL SETUP AND RESULTS

In this section, we will first go through the detailed discussion of the dataset that is created and used in our methodology. Also, we discuss the setup of the final experiment for this research and the results that we have calculated after applying the proposed architecture in the Section IV. We ran our experiments on a high-end system using the NVIDIA Tesla P4 graphics card, along with Intel Xeon Silver 4116 processor which runs 112 cores and 24 Threads. The experiment also takes advantage of 250 GB of RAM.

A. Dataset

To carry out the task of function identification, our first task would be choosing the Android applications dataset that will be reverse-engineered into opcodes and later preprocessed. Since binary code analysis has many fields and one of them is malware detection, we have used Drebin [26] which is the most commonly used dataset in malware detection research community specifically in deep learning. The dataset contains Android application samples for the past 2 years and out of the repository of malware apps available, we are going to utilize only 100 of them in our case. Each of these applications has been tested through several authenticated Antivirus Softwares for surety. Every malware Android app in the dataset belongs to one of the 20 families which are listed on the site in detail, but our focus will be using them for the accurate function

identification and boundary detection for now. The output tensor we get after preprocessing can be displayed as:

$$output_tensor = (instructions,\ embedding\ size, classes) \tag{2}$$

where the total number of instructions out of 100 malware applications are 25,967,763 and each of them after preprocessing step are converted to 50 length vector embedding. The dataset is faced with the problem of imbalanced class nature since we are only focusing on the function start boundary.

B. Experimental Setup

To apply the architecture mentioned in Table I, we use the hyperparameters of the model for the experiments as displayed in Table II. These hyperparameter values were tested with many variations throughout the research and the best of all the variations are used.

TABLE II
HYPERPARAMETERS FOR THE MODEL

Stage	Hyperparameter	Value
Initialization	Weights	Xavier (Default)
Activation Function	ReLU	Alpha=0.0 (Default)
Regularization	Batch Normalization	Mean=0, Std_dev=1
Training	Epochs	50
	Batch Size	2048
	Loss Function	Binary Cross Entropy
	Optimizer	Adam
Model Parameters	Total Parameters	56,446
	Trainable Parameters	56,328
	Non-Trainable Parameters	118

In the training phase, we have used 80% of the dataset instructions with their ground truths out of approx. 25 million instructions. The model represented in Section IV-D is being deployed using Keras deep learning framework. Along with Keras, we are using Tensorflow as backend support to use the capabilities of Nvidia-GPU [27].

1) Input: To evaluate the performance of CNN pipeline developed in Section IV-D we have given opcode embeddings vectors from the total 256 possible opcode occurrences. Model architecture as well as hyperparameters as shown in Table I and Table II are the same for all training phases.

C. Evaluation

The very next and important step after training of the model is to evaluate the model with the right evaluation metrics. For this crucial part, we will be using Precision, Recall, and F1 score [28] as the model performance measures. Accuracy is not mentioned here since the nature of our dataset is skewed and it thus affects the decision process for function identification.

1) Precision: Precision is the quantitative relation of correctly predicted positive observations to the total predicted positive observations. Precision can be indicated as:

$$Precision = \frac{TP}{TP + FP} \tag{3}$$

2) Recall: Recall is the proportion of correctly predicted positive observations with respect to all observations in actual class. It can be mathematically shown as:

$$Recall = \frac{TP}{TP + FN} \tag{4}$$

3) F1 Score: The metrics described in Equation 3 and 4 are usually combined through F1 score, which is the harmonic mean of recall and precision, and it is displayed as:

$$F1Score = \frac{2 * (Recall * Precision)}{Recall + Precision} \tag{5}$$

D. Results

In the following section, we discuss the primary aspects of the experiment which are the results obtained by the testing of our model. As discussed in V-A we are using Android Malware applications and after the preprocessing of the original dataset, we obtain opcode vector representations along with the labels as 1 and 0. Due to the limitations of the GPU, we decided to proceed with the present model parameters and layer descriptions to avoid any computation failures and memory loads.

1) Evaluation Metrics: In this section, we are doing the evaluation of function start boundary detection on the instructions dataset as shown in Table III. For evaluation we are using Precision, Recall, and F1 score as explained in Equations 3, 4 and 5.

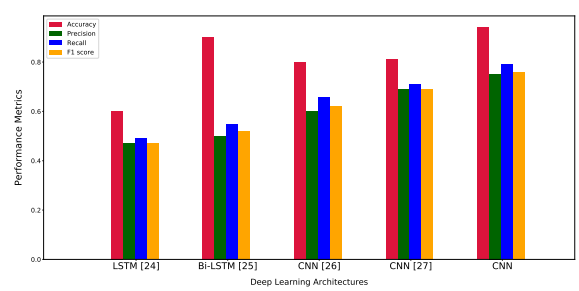

Fig. 5. Imbalanced classes against bytecode instructions

As we can see in Table III and Figure 5, we have implemented multiple versions and variations of the LSTM and CNN models to solve the function start boundary problem, but the performance was not satisfactory. The final row in Table III displays the latest implementation of CNN deep learning architecture that has achieved better results which are shown using the mentioned evaluation metrics.

The graph in Figure 6 represents the drastic increase in the accuracy of the model within half of the epoch. A similar pattern can be followed within the validation steps as well which shows that using accuracy as the only metric in this problem is not the viable option. This is all due to the nature of imbalanced classes in the dataset.

TABLE III
PRECISION, RECALL, AND F1 SCORE OF THE PROPOSED METHOD USING CNN ON ANDROID INSTRUCTIONS OPCODE.

	Accuracy	**Precision**	**Recall**	**F1 Score**
LSTM for IMDB Sentiment Classification [29]	0.60	0.47	0.49	0.47
Bi-LSTM for Function Identification [30]	0.90	0.50	0.55	0.52
CNN for 1D Sentence Classification [31]	0.80	0.60	0.66	0.62
CNN for Text Classifciation [32]	0.81	0.69	0.71	0.69
CNN	**0.94**	**0.75**	**0.79**	**0.76**

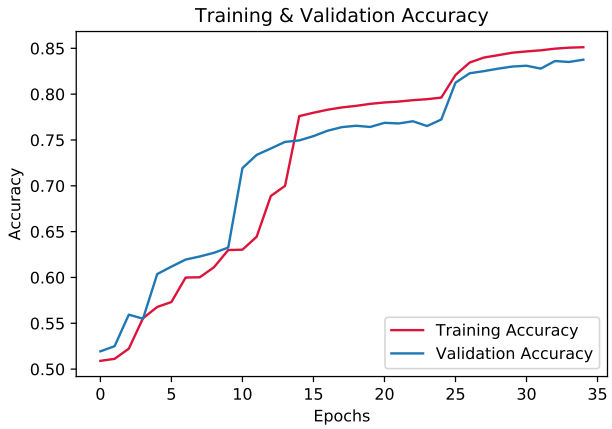

Fig. 6. Training vs Validation over the epochs

VI. DISCUSSION

The main focus of this research was to bring the problem of accurate function boundary detection in the area of Android security using Convolutional Neural Networks and the designed pipeline. We did so by using Drebin Malware Android applications. The whole process of solving this research idea is filled with challenges and difficulties. The first and foremost challenge which persisted in the entire research was dealing with the imbalanced classes which are assigned to instructions captured from malware applications. Currently we have limited the scope of predicting function start boundary only; and although the labeling is enough for the given problem statement, it does lead to model overfitting one label only.

To solve the current problem in the dataset, we could apply techniques such as upsampling and downsampling the dataset values according to the class labels, but that would not be practical in the real world scenario. Therefore, after trying multiple testings with the input layer we finally settled with the Chars2vec vector representation of each opcode. This helps in the initialization of learning weights in the training model. Another thing that helped was to focus on sigmoid activation function on the output layer instead of softmax function, together with the binary cross entropy as opposed to categorical cross entropy. The experiment setup is discussed in detail in the Section V-B.

VII. RELATED WORK

A. Function identification

Various previous research has tried the function identification task, varying from signature-based systems to approaches using machine learning.

Many software solutions are built specifically for binary analysis task contribute to function identification as part of their process, using comparatively simple heuristics or hand-crafted signatures. **Dyninst** [33] and IDA Pro are some examples of all. Dyninst [33] also provides tools, such as unstrip [34], to recognize functions in binaries without having any of the debug information. Potential functions in the text section are identified within the tool using the hex pattern 0x55 representing *push %ebp*. Initially, Dyninst will begin at the entry address and traverse through inter and intra-procedural control flow. The algorithm will search between functions and check if the required assembly instruction is present. This does not perform well across operating systems and various optimizations.

IDA using heuristics and FLIRT [35] technique strives to assist security researchers to recover procedural details. However, updating the signature database needs an amount of manual work that does not scale. Furthermore, since FLIRT uses a pattern matching algorithm to search for signatures, small differences in libraries such as dissimilar compiler optimizations or the use of a compiler versions stop FLIRT from recognizing accurate functions in a disassembled program. The Binary Analysis Platform (BAP) [36] also attempts to present a sound identification of functions using custom-written signatures.

Nucleus [37] is a concurrent work that is also based on static analysis. Nucleus relies on control-flow analysis to infer inter-procedural edges and function starts. It relies on control-flow analysis to infer inter-procedural edges and function starts. They evaluated the model on a diverse set of 476 binaries, including binaries compiled with GCC, Clang, and Visual Studio.

BCD [38] on the other hand, proposes a novel static approach for decomposing an executable into components. Their approach is independent of the compiler applies to both object-oriented and procedural programs. The main theme in their approach is to examine three key properties (code locality,

data references and function calls) that contain useful information regarding the original code components throughout the compilation process. The properties are shown in the structure of a decomposition graph, and a graph-theoretic clustering algorithm is used to identify the components.

MobileFindr [39] runs on mobile devices and reduces many prevalent anti-reversing techniques by extracting function execution behaviors via dynamic instrumentation, then characterizing functions with collected behaviors and performing function matching through distance calculation. MobileFindr has been evaluated using real-world high ranked mobile frameworks and applications.

One way of recognizing function boundaries is to automatically digest main patterns from the data. For example, innovative work by **Rosenblum et al.** [4] suggested binary function start identification as a supervised machine learning classification problem. The model starts identification as a Conditional Random Field (CRF) where binary offsets and a number of selected patterns appear in the CRF. Since typical inference methods for CRF on large and highly-connected graphs are costly, Rosenblum et al. adopted feature selection and rough inference to speed up their model. Nevertheless, they required 150 compute days in 2008 only for the feature selection part on 1,171 binaries. Rosenblum et al. first shaped the function identification problem as a task for machine learning. They mixed a logistic regression classifier that uses short patterns with a conditional random field to impose some structure between predictions for related instructions.

Bao et al. [2] established BYTEWEIGHT which also targets function identification using supervised learning by using weighted prefix trees. It requires much less computation than Rosenblum et al.'s approach to train with high accuracy. This technique automatically learns key features for recognizing functions and can therefore easily be adapted to new compilers, different platforms, and new optimizations. BYTEWEIGHT was evaluated against three well-known tools that perform function identification: Dyninst, BAP, and IDA. Their dataset comprises 2,200 binaries created using two different operating systems, three unrelated compilers, and four, unlike optimization levels. In their experiments with 2,200 binaries, BYTEWEIGHT missed 44,621 functions compared to the 266,672 functions missed by the industry-leading tools like IDA. Also, BYTEWEIGHT had a count of 43,992 misidentified functions while IDA had 459,247 misidentified functions in comparison.

Richard Shin et al. [5] proposed recurrent neural networks [16] to identify functions in binaries. Their model had greater accuracy and efficiency than the state-of-the-art machine-learning-based method. Dataset consists of 2,064 binaries; the one used by BYTEWEIGHT for the purpose of evaluation and comparison of results. The only limitation it has was no work has been done on Android application binaries for the same task.

VIII. Conclusion

The main goal of this research was to introduce the area of accurate function boundary detection from Android binaries/applications such that it can help in the binary analysis and all the related security fields. We have achieved good results after finding the optimal hyperparameters but there is room for improvement. In addition, we have not added any comparative analysis table with the previous recent works since it has not been done in the Android security before, as opposed to computer binaries which have been explored in detail.

The research can be further continued by trying various modifications to the deep learning models. This paper has only tested the LSTM and Bi-directional LSTM model and they did not perform according to the requirement. There are alternatives such as GANs, Autoencoders, and even the other variants of the CNN model that can be tested to see how it works against the dataset that we have generated. The main challenge of this research is the dataset creation from scratch and therefore it escapes the standards with which we can compare our results with. Also, there is very little correlation or connection found between the bytecode instructions and the function boundary classes, which makes is further hard to train any kind of deep learning models. Furthermore, once we are able to finalize the actual shape of a dataset, we can add the labels for function end boundary as well so that the model becomes able to extract all functions from any Android binary which is the primary theme of this project. A whole other dataset on function level could be generated which can be labeled and trained to identify malicious activity on function level instead of an application level.

References

[1] J. Menon, C. Hauser, and Y. Shoshitaishvili, "A binary analysis approach to retrofit security in input parsing routines," 2018. [Online]. Available: https://ieeexplore.ieee.org/document/8424665/

[2] T. Bao, J. Burket, M. Woo, R. Turner, and D. Brumley, "BYTEWEIGHT: Learning to recognize functions in binary code," pp. 845–860, 2014. [Online]. Available: https://www.usenix.org/conference/usenixsecurity14/technical-sessions/presentation/bao

[3] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. The MIT Press, 2016.

[4] N. Rosenblum, X. Zhu, B. Miller, and K. Hunt, "Learning to analyze binary computer code," pp. 798–804, 2008. [Online]. Available: http://dl.acm.org/citation.cfm?id=1620163.1620196

[5] E. C. R. Shin, D. Song, and R. Moazzezi, "Recognizing functions in binaries with neural networks," pp. 611–626, 2015. [Online]. Available: https://www.usenix.org/conference/usenixsecurity15/technical-sessions/presentation/shin

[6] Z. Yang, Z. Hu, Y. Deng, C. Dyer, and A. J. Smola, "Neural machine translation with recurrent attention modeling," *CoRR*, vol. abs/1607.05108, 2016. [Online]. Available: http://arxiv.org/abs/1607.05108

[7] A. Karpathy, G. Toderici, S. Shetty, T. Leung, R. Sukthankar, and F. Li, "Large-scale video classification with convolutional neural networks," in *2014 IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2014, Columbus, OH, USA, June 23-28, 2014*, 2014, pp. 1725–1732. [Online]. Available: https://doi.org/10.1109/CVPR.2014.223

[8] "Dalvik bytecode," 2019. [Online]. Available: https://source.android.com/devices/tech/dalvik/dalvik-bytecode

[9] J. Bergstra and Y. Bengio, "Random search for hyper-parameter optimization," *J. Mach. Learn. Res.*, vol. 13, pp. 281–305, Feb. 2012. [Online]. Available: http://dl.acm.org/citation.cfm?id=2188385.2188395

[10] L. Bottou, "Large-scale machine learning with stochastic gradient descent," in *in COMPSTAT*, 2010.

[11] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Comput.*, vol. 9, no. 8, pp. 1735–1780, Nov. 1997. [Online]. Available: http://dx.doi.org/10.1162/neco.1997.9.8.1735

[12] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "Imagenet classification with deep convolutional neural networks," in *Proceedings of the 25th International Conference on Neural Information Processing Systems - Volume 1*, ser. NIPS'12. USA: Curran Associates Inc., 2012, pp. 1097–1105. [Online]. Available: http://dl.acm.org/citation.cfm?id=2999134.2999257

[13] J. Pennington, R. Socher, and C. D. Manning, "Glove: Global vectors for word representation," in *In EMNLP*, 2014.

[14] T. Mikolov, I. Sutskever, K. Chen, G. S. Corrado, and J. Dean, "Distributed representations of words and phrases and their compositionality," in *Advances in Neural Information Processing Systems 26*, C. J. C. Burges, L. Bottou, M. Welling, Z. Ghahramani, and K. Q. Weinberger, Eds. Curran Associates, Inc., 2013, pp. 3111–3119. [Online]. Available: http://papers.nips.cc/paper/5021-distributed-representations-of-words-and-phrases-and-their-compositionality.pdf

[15] "Character-based word embeddings model based on rnn," 2019. [Online]. Available: https://github.com/IntuitionEngineeringTeam/chars2vec

[16] L. C. Jain and L. R. Medsker, "Recurrent neural networks: Design and applications," 1999.

[17] "Keras conv1d," https://keras.io/layers/convolutional/, 2015.

[18] K. He, X. Zhang, S. Ren, and J. Sun, "Delving deep into rectifiers: Surpassing human-level performance on imagenet classification," *CoRR*, vol. abs/1502.01852, 2015. [Online]. Available: http://arxiv.org/abs/1502.01852

[19] D. Scherer, A. Müller, and S. Behnke, "Evaluation of pooling operations in convolutional architectures for object recognition," in *Proceedings of the 20th International Conference on Artificial Neural Networks: Part III*, ser. ICANN'10. Berlin, Heidelberg: Springer-Verlag, 2010, pp. 92–101. [Online]. Available: http://dl.acm.org/citation.cfm?id=1886436.1886447

[20] J. Han and C. Moraga, "The influence of the sigmoid function parameters on the speed of backpropagation learning," in *Proceedings of the International Workshop on Artificial Neural Networks: From Natural to Artificial Neural Computation*, ser. IWANN '96. London, UK, UK: Springer-Verlag, 1995, pp. 195–201. [Online]. Available: http://dl.acm.org/citation.cfm?id=646366.689307

[21] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," in *Proceedings of the 32Nd International Conference on International Conference on Machine Learning - Volume 37*, ser. ICML'15. JMLR.org, 2015, pp. 448–456. [Online]. Available: http://dl.acm.org/citation.cfm?id=3045118.3045167

[22] R. Hecht-Nielsen, "Neural networks for perception (vol. 2)," H. Wechsler, Ed. Orlando, FL, USA: Harcourt Brace & Co., 1992, ch. Theory of the Backpropagation Neural Network, pp. 65–93. [Online]. Available: http://dl.acm.org/citation.cfm?id=140639.140643

[23] S. Mannor, D. Peleg, and R. Rubinstein, "The cross entropy method for classification," in *Proceedings of the 22Nd International Conference on Machine Learning*, ser. ICML '05. New York, NY, USA: ACM, 2005, pp. 561–568. [Online]. Available: http://doi.acm.org/10.1145/1102351.1102422

[24] J. H. Friedman, "Stochastic gradient boosting," *Comput. Stat. Data Anal.*, vol. 38, no. 4, pp. 367–378, Feb. 2002. [Online]. Available: http://dx.doi.org/10.1016/S0167-9473(01)00065-2

[25] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," *CoRR*, vol. abs/1412.6980, 2014. [Online]. Available: http://arxiv.org/abs/1412.6980

[26] D. Arp, M. Spreitzenbarth, H. Gascon, and K. Rieck, "Drebin: Effective and explainable detection of android malware in your pocket," 2014.

[27] J. Nickolls, I. Buck, M. Garland, and K. Skadron, "Scalable parallel programming with cuda," *Queue*, vol. 6, no. 2, pp. 40–53, Mar. 2008. [Online]. Available: http://doi.acm.org/10.1145/1365490.1365500

[28] C. Goutte and E. Gaussier, "A probabilistic interpretation of precision, recall and f-score, with implication for evaluation," in *Proceedings of the 27th European Conference on Advances in Information Retrieval Research*, ser. ECIR'05. Berlin, Heidelberg: Springer-Verlag, 2005, pp. 345–359. [Online]. Available: http://dx.doi.org/10.1007/978-3-540-31865-1_25

[29] "Lstm model on the imdb sentiment classification task," 2019. [Online]. Available: https://github.com/keras-team/keras/blob/master/examples/imdb_lstm.py

[30] "Trains a bidirectional lstm on the imdb sentiment classification task." 2019. [Online]. Available: https://github.com/keras-team/keras/blob/master/examples/imdb_bidirectional_lstm.py

[31] N. Janakiev, "Practical text classification with python and keras," 2019. [Online]. Available: https://realpython.com/python-keras-text-classification

[32] B. V. Oswal, "Cnn-text-classification-keras," https://github.com/bhaveshoswal/CNN-text-classification-keras, 2016.

[33] L. C. Harris and B. P. Miller, "Practical analysis of stripped binary code," *SIGARCH Comput. Archit. News*, p. 2005.

[34] "Unstrip." [Online]. Available: http://www.paradyn.org/html/tools/unstrip.html

[35] "Ida flirt technology." [Online]. Available: https://www.hex-rays.com/products/ida/tech/flirt/in_depth.shtml

[36] D. Brumley, I. Jager, T. Avgerinos, and E. J. Schwartz, "Bap: A binary analysis platform," in *Proceedings of the 23rd International Conference on Computer Aided Verification*, ser. CAV'11. Berlin, Heidelberg: Springer-Verlag, 2011, pp. 463–469. [Online]. Available: http://dl.acm.org/citation.cfm?id=2032305.2032342

[37] D. Andriesse, A. Slowinska, and H. Bos, "Compiler-Agnostic Function Detection in Binaries," in *Proceedings of the 2nd IEEE European Symposium on Security and Privacy (EuroS&P'17)*. Paris, France: IEEE, April 2017.

[38] V. Karande, S. Chandra, Z. Lin, J. Caballero, L. Khan, and K. Hamlen, "Bcd: Decomposing binary code into components using graph-based clustering," in *Proceedings of the 2018 on Asia Conference on Computer and Communications Security*, ser. ASIACCS '18. New York, NY, USA: ACM, 2018, pp. 393–398. [Online]. Available: http://doi.acm.org/10.1145/3196494.3196504

[39] Y. Liao, R. Cai, G. Zhu, Y. Yin, and K. Li, "Mobilefindr: Function similarity identification for reversing mobile binaries," in *ESORICS (1)*, ser. Lecture Notes in Computer Science, vol. 11098. Springer, 2018, pp. 66–83.